

Day 5 – Searching, Sorting & Java Classes

Linear & Binary Search, Bubble & Insertion Sort, Constructors

1. Searching and Sorting: Overview

Searching techniques are mainly divided into two types: **Linear Search** and **Binary Search**.

Linear Search

A linear search is a process of searching, one by one, through elements to check whether an element exists or not.

Basic idea: scan every element from left to right and compare it with the target.

```
arr = [10, 20, 30, 40, 50]    // indices: 0, 1, 2, 3, 4
t   = 50
```

```
for (i = 0; i < n; i++) {
    if (arr[i] == t) {
        print("True");
    }
}
print("False");
```

Program 1: Linear Search

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int n = arr.length;

        Scanner sc = new Scanner(System.in);
        int target = sc.nextInt();
        int found = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] == target) {
                System.out.print("True");
                found = 1;
                break;
            }
        }

        if (found == 0) {
            System.out.print("False");
        }
    }
}
```

2. Binary Search

Binary search repeatedly divides a **sorted** array in half, comparing the middle element with the target to decide which half to search next.

Program 2: Binary Search

Trace for target = 20 on {10,20,30,40,50}: low=0, high=4 → mid=2, arr[2]=30 (30>20) → high=1; low=0, high=1 → mid=0, arr[0]=10 (10<20) → low=1; low=1, high=1 → mid=1, arr[1]=20 → match found, prints True.

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int n = arr.length;

        Scanner sc = new Scanner(System.in);
        int low = 0;
        int high = n - 1;
        int target = sc.nextInt();

        while (low <= high) {
            int mid = (low + high) / 2;

            if (arr[mid] == target) {
                System.out.print("True");
                return;
            }
            if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }

        System.out.print("False");
    }
}
```

3. Sorting: Bubble Sort

Bubble sort is a sorting algorithm that compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest remaining element moves to the end.

Program 3: Bubble Sort

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {40, 10, 20, 50, 25};
        int n = arr.length;
```

```

        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }

        System.out.print(Arrays.toString(arr));
    }
}

```

Output:

[10, 20, 25, 40, 50]

4. Sorting: Insertion Sort

Insertion sort is an algorithm that takes one element at a time and inserts it into its correct position among the already-sorted elements.

Program 4: Insertion Sort

```

import java.util.*;

class Main {
    public static void main(String[] args) {
        int[] arr = {10, 5, 40, 12, 15};
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;

            while (j >= 0 && arr[j] > key) {
                arr[j + 1] = arr[j];
                j--;
            }
            arr[j + 1] = key;
        }

        System.out.print(Arrays.toString(arr));
    }
}

```

5. Class, Object and Constructor in Java

A **constructor** is a special member of a class that is automatically called once an object is created.

Rules for a Constructor

- The constructor name must be the same as the class name.
- It has no return type – not even void.

Program 5: Movie Class – Default Constructor

```
class Movie {
    String name;
    int budget;

    void display() {
        System.out.println(name);
        System.out.println(budget);
    }
}

class Main {
    public static void main(String[] args) {
        Movie m1 = new Movie();
        m1.name = "with love";
        m1.budget = 100;

        Movie m2 = new Movie();
        m2.name = "Toxic";
        m2.budget = 350;

        System.out.println(m2.name);
        m1.display();
    }
}
```

Output:

```
Toxic
with love
100
```

Program 6: Movie Class – Parameterized Constructor

```
class Movie {
    String name;
    int budget;

    Movie(String name, int budget) {
        this.name = name;
        this.budget = budget;
    }

    void display() {
        System.out.println(name);
        System.out.println(budget);
    }
}

class Main {
```

```
public static void main(String[] args) {  
    Movie m1 = new Movie("Charlie", 100);  
    Movie m2 = new Movie("Interstellar", 200);  
  
    m2.display();  
    m1.display();  
}  
}
```

Output:

```
Interstellar  
200  
Charlie  
100
```